SPECIFICATION

Calypso Terminal API

Reader

Version 3.0.0-SNAPSHOT, 2026-07-20

Warning. The present document cancels and replaces the previous release.

Link and Contact

Website
https://calypsonet.org

Support
support@calypsonet.org

Table of Contents

1. Abstract

This document is the official technical specification of the **Terminal Reader API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way, the interfaces, classes, enumerations, exceptions, structural relationships and behavioural requirements that any conforming implementation of the Terminal Reader API MUST satisfy.

The Terminal Reader API is part of the broader CNA **Terminal API** family and acts as the contract through which terminal applications discover card readers, observe card events, select cards and exchange data with them.

Document Status

Reference	YYMMDD-SP-CNATerminalAPI-Reader
Short name	CNA-TR-API
Version	3.0.0-SNAPSHOT
Revision date	2026-07-20
Editor	Calypso Networks Association
Source repository	https://github.com/calypsonet/calypsonet-terminal-reader-uml-api
Reference license	Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)

The present document specifies the 3.0.0-SNAPSHOT of the Terminal Reader API. It defines a single, coherent baseline against which conforming implementations are evaluated; the **Since** column indicates the version in which each member was introduced.

Revision List

This section lists the high-level changes per version. The complete, fine-grained changelog is maintained in the `CHANGELOG.md` file at the root of the source repository.

Version / Date	Modifications
v3.0.0-SNAPSHOT 2026-07-20	Baseline.

2. Introduction

2.1. Purpose

The Terminal Reader API defines a standardised set of contracts allowing terminal applications to interact with card readers and the cards presented to them, independently of:

- the underlying reader hardware (contact, contactless, virtual, etc.),
- the card technology family (ISO/IEC 7816-4, ISO/IEC 14443, proprietary),
- the implementation technology of the application (programming language, runtime).

Conforming implementations of this specification MUST allow extension modules (e.g. card-specific APIs such as the Calypso Card API) to plug in transparently through the Service Provider Interface (SPI) mechanism defined in `reader.selection.spi` and `reader.transaction.spi`.

2.2. Scope

This specification covers:

- the discovery and basic interrogation of card readers,
- the observation of reader events related to card presence and selection,
- the configuration of card detection (RF technologies, detection mode, ECP frame),
- the description and orchestration of card selection scenarios, including multi-channel scenarios,
- the abstraction of the physical and logical communication channels established with cards,
- the contract surface required by card extension modules to interoperate with reader implementations,
- the management of single-channel and multi-channel card transactions.

The following topics are **out of scope**:

- the wire-level protocols used between reader and card (e.g. APDU framing, ISO/IEC 14443 anti-collision),
- the cryptographic operations performed by or on behalf of the card,
- the lifecycle and packaging of reader drivers,
- the user interface presented by the terminal application.

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface and class defined in Chapter 5 with the operations and semantics prescribed in this document,
2. it raises exceptions exclusively of the types defined in Section 5.5 for the situations described,
3. it preserves the structural relationships described in Chapter 4.

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

In conformance with RFC 2119, **SHALL** is equivalent to **MUST**; this specification uses **MUST** exclusively in order

to avoid ambiguity.

2.4. Document Conventions

2.4.1. Naming

Type names follow `UpperCamelCase`; operation, parameter and enumeration-member names follow `lowerCamelCase` except for enumeration values which use `UPPER_SNAKE_CASE`. Names defined by this specification are reproduced as-is in the UML class diagram (Section 4.6).

2.4.2. Language-agnostic types

The following symbolic type names are used in operation signatures and are mapped, in each binding, to the closest equivalent native type. Implementations MUST preserve the semantic intent rather than the syntactic form.

Symbolic type	Description	Typical bindings
`string`	Sequence of characters, UTF-8 encoded by default.	`String`, `string`, `str`.
`boolean`	Two-state truth value.	`boolean`, `bool`.
`integer`	Signed 32-bit integer.	`int`, `Int32`.
`long`	Signed 64-bit integer.	`long`, `Int64`.
`byte array`	Ordered sequence of octets.	`byte[]`, `[u8]`, `bytes`.
`list<T>`	Ordered collection of `T` values, may contain duplicates.	`List<T>`, `Vec<T>`.
`set<T>`	Unordered collection of unique `T` values.	`Set<T>`, `HashSet<T>`.
`map<K,V>`	Associative array binding keys of type `K` to values of type `V`.	`Map<K,V>`, `Dictionary<K,V>`.
`void`	Indicates that an operation returns no value.	`void`, `Unit`, `None`.

2.4.3. Type tables

Each interface, class, enumeration and exception defined in this document is described by a table of the form:

Name	The type's normalized name.
Kind	The category of the type (interface, class, enumeration, exception, etc.).
Namespace	The namespace in which the type is defined.
Generics	The generic type parameter of this type, when applicable.
Extends	The type extended by this type, when applicable.
Implements	The interface implemented by this type, when applicable.
Since	The version of the API in which the type was introduced.
Purpose	A normative description of the type's responsibility.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

The **Purpose** row states, in one or two sentences, the responsibility of the type and the way an instance is obtained. It is meant to be scanned, not read: any content that requires structure or depth—rules, lists, notes, value tables or examples—is placed as prose below the table.

2.4.4. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation's language-agnostic signature.
Since	The version of the API in which the operation was introduced.
Description	A normative description of the operation's behaviour.
Parameters	A description of each input parameter.
Returns	A description of the returned value, if any.
Throws	A list of exceptional conditions, each mapped to a specific exception type.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

2.4.5. Operation contracts

To avoid repetition, the following rules apply to every operation table in Chapter 5 and are therefore not restated for each operation:

- **Non-`null` arguments.** Unless explicitly stated otherwise, every input parameter MUST be non-`null`. An implementation MUST raise `IllegalArgumentException` if a `null` value is supplied. This implicit `null` check is not repeated in the **Throws** row, which states `None.` when no other exception can occur.
- **Non-`null` results.** Unless the return type is marked nullable (`T?`) or the **Returns** row states otherwise, an operation returns a non-`null` value.
- **Collections.** Unless the return type is marked nullable (`T?`), an operation whose return type is a collection (e.g. `list`, `set`, `map`) returns an empty collection rather than `null`.
- **Fluent composition.** Builder-style operations return the current instance so that calls can be chained.

2.4.6. Concurrency

Unless explicitly stated otherwise, the stateful types defined by this specification are **not** thread-safe. A given instance MUST be accessed from a single thread at a time; concurrent use of the same instance without external synchronisation results in undefined behaviour. Distinct instances MAY be used concurrently.

2.5. References and Resources

2.5.1. Calypso References

References to CNA Terminal APIs designate the indicated **major** version; any later release within the same major is backward-compatible per that API's evolution policy.

Reference	Document
[CNA-TD-API]	*CNA Terminal API — Definitions* (SP-CNATerminalAPI-Definitions), version 1.0, Calypso Networks Association. Provides the `RfTechnology` and `CardType` enumerations referenced by this specification.

2.5.2. Normative References

Reference	Document
[RFC 2119]	*Key words for use in RFCs to Indicate Requirement Levels*, S. Bradner, March 1997.

Reference	Document
[RFC 8174]	*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, B. Leiba, May 2017.
[ISO/IEC 7816]	*Identification cards — Integrated circuit cards*.
[ISO/IEC 14443]	*Identification cards — Contactless integrated circuit cards — Proximity cards*.

2.5.3. External Resources

Resource	Link
Calypso Networks Association	https://calypsonet.org
Terminal APIs website	https://terminal-api.calypsonet.org
Terminal APIs documentation	https://docs.terminal-api.calypsonet.org

3. Glossary and Acronyms

3.1. Glossary

Term	Definition
Card	Any physical or virtual smart card that can be presented to a reader for selection and dialog.
Card Reader	Hardware or software component capable of detecting cards and exchanging data with them.
Card Selection	Sequence of operations used to identify, filter and prepare a card for application-level dialog.
Logical Channel	Software-level communication path established with a card after a successful selection.
Basic Channel	The default logical channel (channel `0`) of an ISO/IEC 7816-4 card.
Multi-Channel	Capability of an ISO/IEC 7816-4 card to host several active logical channels in parallel.
Physical Channel	Hardware-level communication path between the reader and the card.
Observable Reader	Reader implementation able to notify a registered event handler asynchronously of card-related events.
Power-on Data	Data returned by the reader at the moment a card is detected (e.g. ATR, virtual ATR, anti-collision data).
Card Application	Application Dedicated File (DF) selected on a card, identified by its AID, as defined in ISO/IEC 7816-4.
AID	Application IDentifier as defined in ISO/IEC 7816-4 (5 to 16 bytes).
APDU	Application Protocol Data Unit, the message format used between the terminal and the card, as defined in ISO/IEC 7816-3.
ECP	Enhanced Contactless Polling, an extended polling mechanism enabling fast card detection in mobile transit deployments.
RF Technology	Radio-frequency communication technology used by a contactless reader to detect a card, as defined in ISO/IEC 14443.
Card Type	Identified product or transport-protocol family of a detected card.
SPI	Service Provider Interface — a set of interfaces designed to be implemented by an extension module.

3.2. Acronyms

Abbreviation	Expansion
AID	Application IDentifier
APDU	Application Protocol Data Unit
ATR	Answer To Reset
CNA	Calypso Networks Association
DF	Dedicated File
ECP	Enhanced Contactless Polling
FCI	File Control Information
FCP	File Control Parameters
FMD	File Management Data
ISO	International Organization for Standardization
RF	Radio Frequency
RFC	Request For Comments

Abbreviation	Expansion
SPI	Service Provider Interface
UML	Unified Modelling Language

4. Architectural Overview

4.1. Functional positioning

The Terminal Reader API sits between the terminal application and the reader drivers, exposing a stable contract on each side:

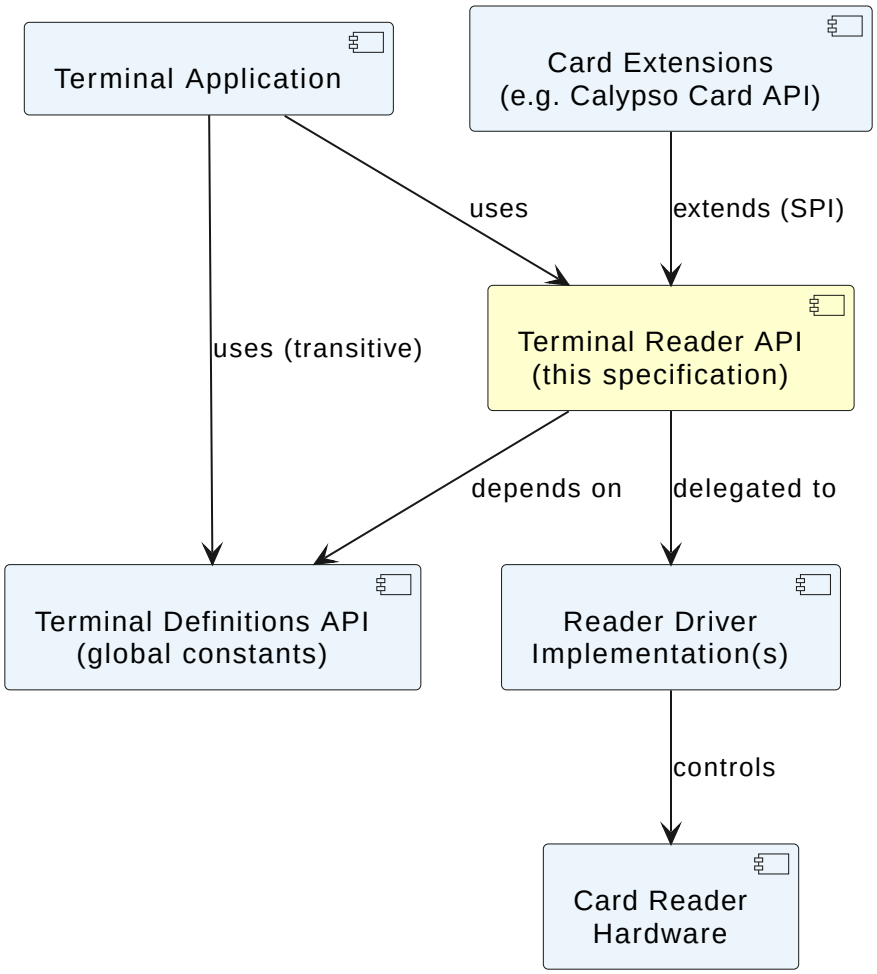

Figure 1. Terminal Reader API — Architecture Overview

4.2. Logical namespaces

Namespace	Role	Summary
`reader`	Public API	Entry points of the API and the abstractions used to manipulate readers and the events they produce; exposes `CardReaderProvider` to discover and access the readers available in the execution environment.
`reader.spi`	SPI	Contracts implemented by the application (as opposed to the reader implementation) when readers are observed.
`reader.selection`	Public API	Orchestration of card selection scenarios: the central `CardSelectionManager`, the family of selectors, the scenario-execution policies and the structures exposing selection results.
`reader.selection.spi`	SPI	Contracts implemented by card extensions to plug into the selection machinery.

Namespace	Role	Summary
`reader.transaction.spi`	SPI	Common contract shared by all card transaction managers exposed by card extensions; its three interfaces form a hierarchy letting each extension anchor its transaction manager at the capability level matching its card model (non-ISO, optionally multi-channel, intrinsically multi-channel).

4.3. Multi-channel design

The Terminal Reader API explicitly supports ISO/IEC 7816-4 cards that host several active logical channels in parallel. The design follows a three-tier hierarchy in the `reader.transaction.spi` namespace:

1. `CardTransactionManager` — root contract common to every transaction manager, regardless of channel model.
2. `IsoCardTransactionManager` — intermediate contract for ISO/IEC 7816-4 cards for which multi-channel is an **optional capability** of the underlying card. It exposes the cast operation `asMultichannelCardTransactionManager` that returns a `MultichannelCardTransactionManager` when the card supports it.
3. `MultichannelCardTransactionManager` — contract for cards that are **intrinsically multi-channel** by design. Consumer APIs whose target cards are always multi-channel SHOULD stereotype their transaction manager directly on this contract.

4.4. Card selection modes

A card selection scenario — prepared through the `CardSelectionManager` — can be operated in two ways, chosen according to the reader capability and the use case.

This section is *informative*. It gives an end-to-end view of the two modes; the normative behaviour of each operation is defined in the `CardSelectionManager` section.

Synchronous (explicit)

The card is already present on a `CardReader`. The application runs the scenario itself with `processCardSelectionScenario` (or `processMultichannelCardSelectionScenario` for multi-channel cards) and obtains the `CardSelectionResult` immediately. The **same entity** manages both the selection and the subsequent card processing.

Asynchronous (scheduled / event-driven)

On an `ObservableCardReader`, the scenario is registered ahead of time with `scheduleCardSelectionScenario` and executed automatically by the reader upon card insertion. The outcome is delivered as a `CardReaderEvent` to the `CardReaderEventHandler` — its type (`CARD_MATCHED` or `CARD_INSERTED`) depending on the `CardPresenceNotificationPolicy` — and decoded with `parseScheduledCardSelectionsResponse`. Here the **reader** drives the selection while the **event handler** performs the card processing.

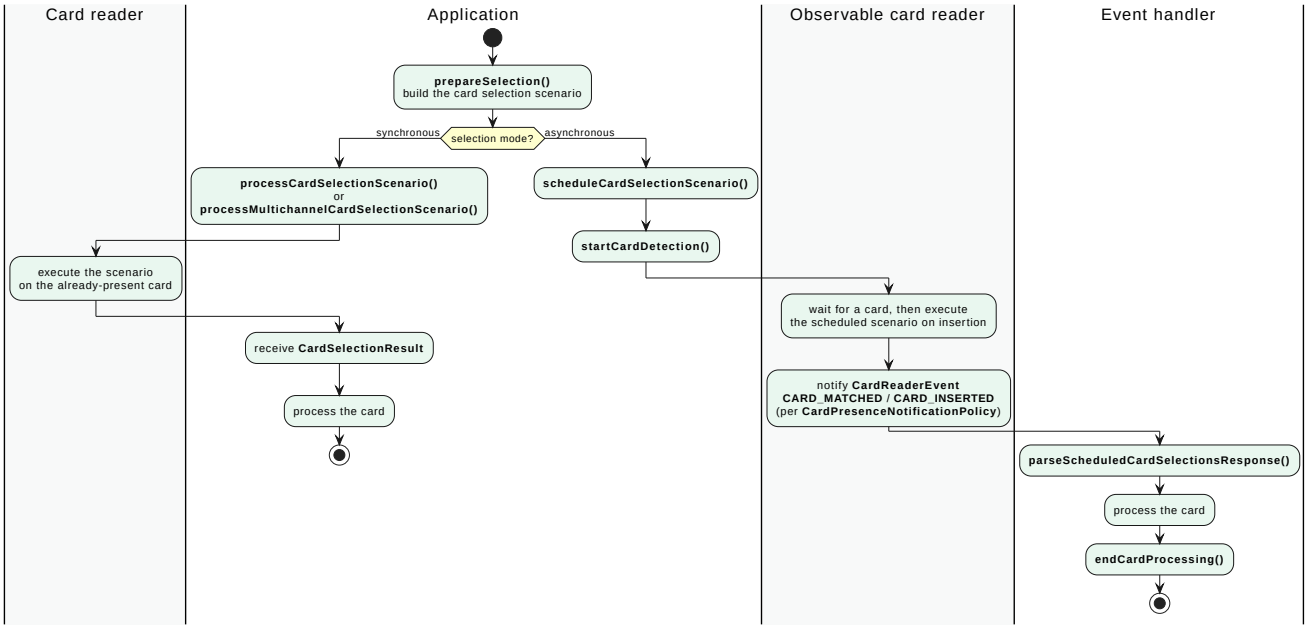

Figure 2. Terminal Reader API — Card Selection Modes Activity Diagram

The runtime execution of the asynchronous mode — the reader states and transitions — is detailed in Reader observation lifecycle.

4.5. Reader observation lifecycle

An `ObservableCardReader` tracks card insertion and removal through an internal state machine. The diagram below gives the integrator a complete view of that lifecycle: how `startCardDetection` / `stopCardDetection` and `endCardProcessing` move the reader between states, when a `CardReaderEvent` is produced, and how the `DetectionMode` and `CardPresenceNotificationPolicy` settings steer the flow.

This section is *informative*. The state and trigger names shown are internal to the implementation and are not part of the normative API surface; they are provided only to explain the observed runtime behavior.

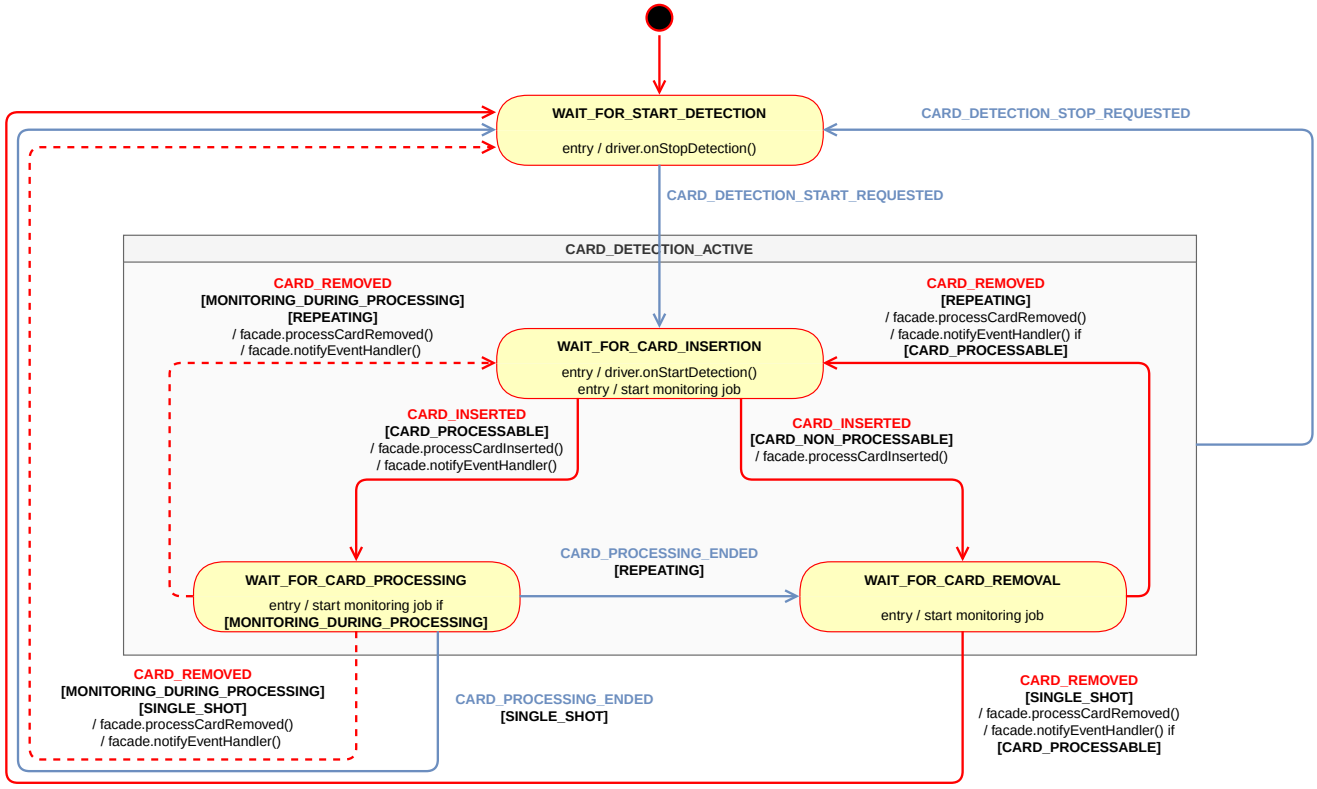

Figure 3. Card Reader Observation — FSM State Diagram

The diagram uses the following notation:

Triggers — events that drive the transitions:

- **Application** triggers (blue transitions in the diagram):
  - `CARD_DETECTION_START_REQUESTED` — the application called `startCardDetection`.
  - `CARD_DETECTION_STOP_REQUESTED` — the application called `stopCardDetection` (factorized on the composite-state boundary).
  - `CARD_PROCESSING_ENDED` — the application called `endCardProcessing`.
- **Hardware** triggers (red transitions in the diagram):
  - `CARD_INSERTED` — the monitoring job detected a card.
  - `CARD_REMOVED` — the monitoring job detected card removal.

Reader capabilities:

- `[MONITORING_DURING_PROCESSING]` — the reader can detect card removal while processing is running, **AND** card processing is executed in a dedicated thread.

Card eligibility:

- `[CARD_PROCESSABLE]` — the inserted card matched at least one case of the card selection scenario, **OR** the notification policy covers every detected card regardless of selection (`CardPresenceNotificationPolicy == ALWAYS`).
- `[CARD_NON_PROCESSABLE]` — the inserted card matched no case of the card selection scenario, **AND** the notification policy is restricted to matched cards (`CardPresenceNotificationPolicy == MATCHED_ONLY`).

Detection mode — the active `DetectionMode`, evaluated when a card-processing cycle ends:

- `[REPEATING]` — detection resumes after each card-processing cycle, ready for the next card.
- `[SINGLE_SHOT]` — detection stops after the first card-processing cycle; a new call to `startCardDetection` is required to detect further cards.

Actions — effect executed on a transition:

- `/ facade.operation()` — a public API call (event production or event-handler notification).
- `/ driver.operation()` — a low-level hardware call (hardware notification).
- `/ start job` — a monitoring job submitted to the background executor.

4.6. UML class diagram

The following UML class diagram provides a visual representation of the types and relationships defined by this specification:

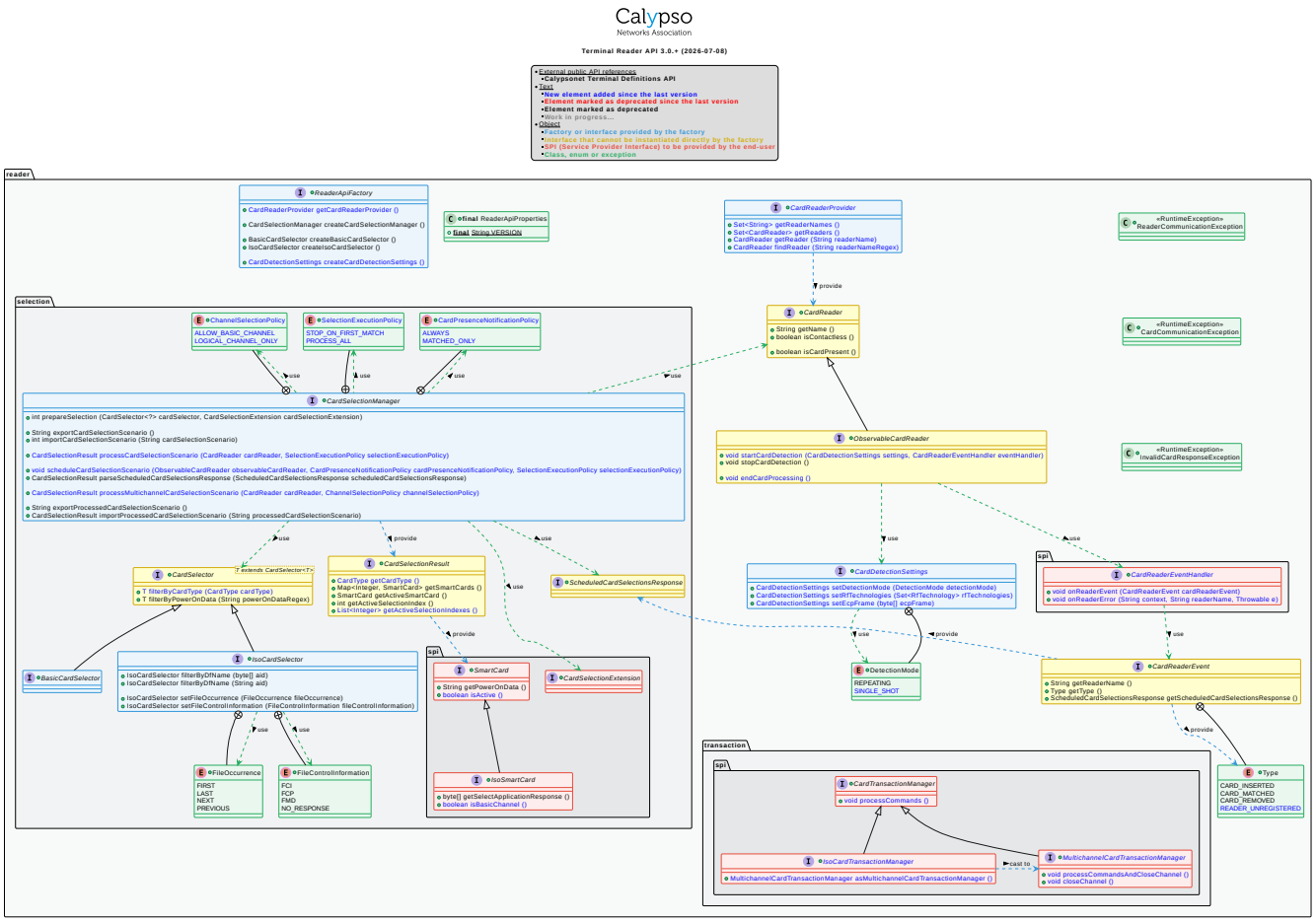

Figure 4. Terminal Reader API — UML Class Diagram

5. API Specification

5.1. Classes

5.1.1. Reader API Properties

Name	`ReaderApiProperties`
Kind	Final class
Namespace	`reader`
Since	1.0
Purpose	Exposes the immutable properties of the Terminal Reader API.

Constants

Name	Type	Since	Description
`VERSION`	`string`	1.0	String representation of the version of the API implemented by the conforming binding (e.g. `"3.0"`). The value is a dotted decimal of the form `MAJOR.MINOR`.

The `ReaderApiProperties` type cannot be instantiated; it exposes only static properties.

5.2. API Interfaces

5.2.1. Basic Card Selector

Name	`BasicCardSelector`
Kind	Interface
Namespace	`reader.selection`
Extends	`CardSelector<BasicCardSelector>`
Since	2.0
Purpose	Basic, technology-agnostic card selector. An instance is obtained via `ReaderApiFactory.createBasicCardSelector`.

`BasicCardSelector` does not add any operation to `CardSelector`. It exists to express the absence of ISO/IEC 7816-4-specific filters at the type level.

The filters carried by a selector restrict the selection process to certain cards. They are all **optional** and MAY be combined. If no filter is specified, any card that responds when presented to the reader is considered selected. Conversely, when one or more filters are defined, the card is not selected as soon as one of them rejects it.

5.2.2. Card Detection Settings

Name	`CardDetectionSettings`
Kind	Interface (builder)
Namespace	`reader`
Since	3.0

Purpose	Carries the configuration of a card detection cycle. An instance is obtained via `ReaderApiFactory.createCardDetectionSettings`.

`CardDetectionSettings` exposes setters in **builder** form: each operation returns the current instance to allow fluent composition. Setters that are not called default to the values specified below.

Default detection mode	`REPEATING`
Default RF technologies	`{ ISO_14443_AB }` (single-element set, see CNA-TD-API)
Default ECP frame	*none*

Set Detection Mode

Signature	`setDetectionMode(detectionMode: DetectionMode) → CardDetectionSettings`
Since	3.0
Description	Sets the **detection mode**, which governs the reader's behaviour once a card-processing cycle has terminated. With `REPEATING`, the reader automatically resumes polling and waits for the next card insertion, enabling continuous, unattended operation. With `SINGLE_SHOT`, the reader stops detection after the current cycle; a new call to `startCardDetection` is required before another card can be detected. When this setter is not called, the detection mode defaults to `REPEATING` (see the defaults above). The full semantics of each value are given by the `DetectionMode` enumeration.
Parameters	`detectionMode` (`DetectionMode`) — the desired detection mode.
Returns	The current instance (`CardDetectionSettings`).
Throws	None.

Set ECP Frame

Signature	`setEcpFrame(ecpFrame: byte array) → CardDetectionSettings`
Since	3.0
Description	Provides the ECP (Enhanced Contactless Polling) frame to emit at polling startup. The frame is treated by this specification as opaque binary data constructed by the application in accordance with the relevant ECP specification and transmitted as-is to the reader. Relevant only on readers supporting ECP. If the underlying reader does not support ECP, the provided frame is silently ignored and a `WARN`-level message is typically logged. When this setter is not called, no ECP frame is emitted (default **none**; see the defaults above).
Parameters	`ecpFrame` — the non-empty ECP frame.
Returns	The current instance (`CardDetectionSettings`).
Throws	`IllegalArgumentException` — if the provided array is empty.

Set RF Technologies

Signature	`setRfTechnologies(rfTechnologies: set<RfTechnology>) → CardDetectionSettings`
Since	3.0

Description	Declares the set of RF technologies the reader activates during polling. The `RfTechnology` enumeration is defined in the Terminal Definitions API (see CNA-TD-API). Not relevant for contact readers; on such readers, this setter is accepted without effect. When this setter is not called, the reader polls the default set `{ ISO_14443_AB }` (see the defaults above).
Parameters	`rfTechnologies` — the non-empty set of RF technologies to activate.
Returns	The current instance (`CardDetectionSettings`).
Throws	`IllegalArgumentException` — if the provided set is empty.

5.2.3. Card Reader

Name	`CardReader`
Kind	Interface
Namespace	`reader`
Since	1.0
Purpose	Represents a card reader capable of driving the underlying hardware to manage card detection.

Get Name

Signature	`getName() → string`
Since	1.0
Description	Returns the name of the reader.
Returns	A non-empty `string` identifying the reader. The value is stable for the lifetime of the reader instance.
Throws	None.

Is Card Present

Signature	`isCardPresent() → boolean`
Since	1.0
Description	Indicates whether a card is currently detected by the reader. What counts as **present** depends on the reader's communication mode (see `isContactless`): on a **contact** reader, a card is present when it is physically inserted in the slot; on a **contactless** reader, a card is present when it lies within range of the antenna (in the RF field).
Returns	`true` if a card is present — inserted for a contact reader, or within the field for a contactless reader — `false` otherwise.
Throws	`ReaderCommunicationException` — if the communication with the reader has failed.

Is Contactless

Signature	`isContactless() → boolean`
Since	1.0
Description	Indicates whether the card communication mode is contactless.
Returns	`true` if the communication mode is contactless, `false` otherwise.
Throws	None.

5.2.4. Card Reader Event

Name	`CardReaderEvent`
Kind	Interface
Namespace	`reader`
Since	1.0
Purpose	Data container describing a change of state observed by a `CardReader`.

A `CardReaderEvent` carries the origin of the event (the reader name), the event type and, when available, the card selection response. The circumstances under which each event type is emitted are described in Reader observation lifecycle.

Get Reader Name

Signature	`getReaderName() → string`
Since	1.0
Description	Returns the name of the reader that generated the event.
Returns	A non-empty `string`.
Throws	None.

Get Scheduled Card Selections Response

Signature	`getScheduledCardSelectionsResponse() → ScheduledCardSelectionsResponse?`
Since	1.0
Description	Returns the response of the selection scenario scheduled on the reader. It is `null` **only** when no selection scenario has been scheduled; in every other case it is non-`null`. When non-`null`, the returned value MUST be passed for interpretation to `CardSelectionManager.parseScheduledCardSelectionsResponse`, called on the `CardSelectionManager` instance that prepared the associated selection scenario, or on another `CardSelectionManager` sharing the same context—that is, one that has previously prepared the same selection scenario.
Returns	The scheduled card selections response (`ScheduledCardSelectionsResponse`), or `null` if no selection scenario has been scheduled.
Throws	None.

Get Type

Signature	`getType() → Type`
Since	1.0
Description	Returns the type of the event.
Returns	A `Type` value.
Throws	None.

5.2.5. Card Reader Provider

Name	`CardReaderProvider`
Kind	Interface

Namespace	`reader`
Since	3.0
Purpose	Standardised discovery and access surface for the card readers available in the execution environment.

The `CardReaderProvider` is obtained from the `ReaderApiFactory`. It offers an implementation-agnostic view over the readers currently registered in the environment. Reader lifecycle (registration and un-registration) is handled by the runtime environment; the provider is a **read-only** view reflecting the readers active at the moment of each call.

Find Reader

Signature	`findReader(readerNameRegex: string) → CardReader?`
Since	3.0
Description	Returns the first reader whose name matches the supplied regular expression. This lookup is convenient when reader names embed variable elements (serial numbers, USB port index, etc.) that the application cannot hard-code.
Parameters	`readerNameRegex` — the regular expression matched against the reader names.
Returns	A `CardReader` whose name matches the expression, or `null` if none matches.
Throws	`IllegalArgumentException` — if the supplied string is not a valid regular expression.

Get Reader

Signature	`getReader(readerName: string) → CardReader?`
Since	3.0
Description	Returns the reader whose name exactly matches the supplied string.
Parameters	`readerName` — the exact name of the reader to retrieve.
Returns	A `CardReader` whose name equals `readerName`, or `null` if no such reader is registered.
Throws	None.

Get Reader Names

Signature	`getReaderNames() → set<string>`
Since	3.0
Description	Returns the names of all readers currently registered in the execution environment.
Returns	A `set` of reader names; empty if no reader is registered.
Throws	None.

Get Readers

Signature	`getReaders() → set<CardReader>`
Since	3.0
Description	Returns all readers currently registered in the execution environment.
Returns	A `set` of `CardReader` references; empty if no reader is registered.
Throws	None.

5.2.6. Card Selection Manager

Name	`CardSelectionManager`
Kind	Interface
Namespace	`reader.selection`
Since	1.0
Purpose	Service responsible for preparing and executing card selection scenarios. An instance is obtained via `ReaderApiFactory.createCardSelectionManager`.

A **card selection scenario** is composed of one or more **selection cases**, each backed by a `CardSelectionExtension`. A selection case targets a specific card and MAY define additional commands to be executed after the successful selection. When a selection case fails, the manager tries the next selection case defined in the scenario, until no further selection case is available. The behaviour of the scenario is governed by the `SelectionExecutionPolicy` passed at execution time:

- with `STOP_ON_FIRST_MATCH`, the manager stops at the first successful selection case;
- with `PROCESS_ALL`, the manager processes every selection case regardless of intermediate successes.

For ISO/IEC 7816-4 multi-channel cards, a dedicated execution method is provided (`processMultichannelCardSelectionScenario`), parameterised by a `ChannelSelectionPolicy`.

The manager provides three flavours of execution: explicit (synchronous) single-channel selection, explicit multi-channel selection, and scheduled (event-driven) selection. It additionally supports **export/import** of the scenario or its result for distributed architectures. See Card selection modes for an end-to-end view of the synchronous and asynchronous flows.

A scenario is considered **prepared** once at least one selection case has been appended via `prepareSelection`. A prepared scenario MAY be executed multiple times; each execution operates on the scenario as it stood at the time of the call.

Export Card Selection Scenario

Signature	`exportCardSelectionScenario() → string`
Since	1.1
Description	Exports the content of the current prepared selection scenario as a `string`. The result MAY be imported into the same or another `CardSelectionManager` via `importCardSelectionScenario`.
Returns	A non-empty `string`.
Throws	None.
See also	`importCardSelectionScenario`

Export Processed Card Selection Scenario

Signature	`exportProcessedCardSelectionScenario() → string`
Since	1.3

Description	Exports the content of the previously processed selection scenario as a `string`. The result MAY be imported through `importProcessedCardSelectionScenario`. **Prerequisite:** the selection scenario MUST have been processed previously via `processCardSelectionScenario`, `processMultichannelCardSelectionScenario` or `parseScheduledCardSelectionsResponse`. **Caution:** if the local environment does not have all the card extensions involved in the scenario, the `CardSelectionResult` produced locally will not contain any active selection. In that case, the processed scenario MUST be exported so that it can be re-imported and interpreted on a manager which has the relevant card extensions.
Returns	A non-empty `string`.
Throws	`IllegalStateException` — if the scenario has not yet been processed or has failed.
See also	`importProcessedCardSelectionScenario`

Import Card Selection Scenario

Signature	`importCardSelectionScenario(cardSelectionScenario: string) → integer`
Since	1.1
Description	Imports a previously exported scenario. The input MUST have been produced by `exportCardSelectionScenario`.
Parameters	`cardSelectionScenario` — the `string` containing the exported card selection scenario.
Returns	The index of the last imported selection in the scenario.
Throws	`IllegalArgumentException` — if the string is malformed.
See also	`exportCardSelectionScenario`

Import Processed Card Selection Scenario

Signature	`importProcessedCardSelectionScenario(`
	`  processedCardSelectionScenario: string`
	`) → CardSelectionResult`

Since	1.3
Description	Imports a previously exported **processed** selection scenario and returns the corresponding result. **Prerequisites:** the input MUST have been produced by `exportProcessedCardSelectionScenario`; the local environment MUST possess every card extension involved; the current manager MUST first have been configured with the same selection scenario as the manager that produced the export.
Parameters	`processedCardSelectionScenario` — the `string` containing the exported processed card selection scenario.
Returns	A `CardSelectionResult`.
Throws	`IllegalArgumentException` — if the string is malformed or contains more selection cases than the current scenario. `InvalidCardResponseException` — if the data returned by the card could not be interpreted.
See also	`exportProcessedCardSelectionScenario`

Parse Scheduled Card Selections Response

Signature	`parseScheduledCardSelectionsResponse(`
	`  scheduledCardSelectionsResponse: ScheduledCardSelectionsResponse`
	`) → CardSelectionResult`

Since	1.0

Description	Analyses the responses provided by a `CardReaderEvent` following the insertion of a card and the execution of the scheduled selection scenario.
Parameters	`scheduledCardSelectionsResponse` (`ScheduledCardSelectionsResponse`) — the card selection scenario execution response.
Returns	A `CardSelectionResult`.
Throws	`InvalidCardResponseException` — if the data returned by the card could not be interpreted.

Prepare Selection

Signature	`prepareSelection(`
	`  cardSelector: CardSelector<?>,`
	`  cardSelectionExtension: CardSelectionExtension`
	`) → integer`

Since	2.0
Description	Appends a selection case to the scenario. The returned index gives the position of the selection in the scenario (`0` for the first case, `1` for the second, etc.) and MUST be used to retrieve the corresponding result in the `CardSelectionResult`.
Parameters	`cardSelector` (`CardSelector`) — the card selector containing the filters to be used to select the card. `cardSelectionExtension` (`CardSelectionExtension`) — the card selection extension to be used to parse the card selection response.
Returns	A non-negative `integer` — the index of the appended selection case.
Throws	None.

Process Card Selection Scenario

Signature	`processCardSelectionScenario(`
	`  reader: CardReader,`
	`  selectionExecutionPolicy: SelectionExecutionPolicy`
	`) → CardSelectionResult`

Since	3.0
Description	Explicitly executes the previously prepared selection scenario against the provided reader, with the requested execution policy, and returns the result.
Parameters	`reader` (`CardReader`) — the reader through which the card is reached. `selectionExecutionPolicy` (`SelectionExecutionPolicy`) — the policy governing the iteration over selection cases.
Returns	A `CardSelectionResult`.
Throws	`ReaderCommunicationException` — if communication with the reader has failed. `CardCommunicationException` — if communication with the card has failed. `InvalidCardResponseException` — if the card returned invalid data during the selection process, or if the status word check is enabled in the card request and the card returned an unexpected code.

Process Multichannel Card Selection Scenario

Signature	`processMultichannelCardSelectionScenario(`
	`  reader: CardReader,`
	`  channelSelectionPolicy: ChannelSelectionPolicy`
	`) → CardSelectionResult`

Since	3.0

Description	Explicitly executes the previously prepared selection scenario in **multi-channel** mode against the provided reader, with the requested channel policy, and returns the result. **Prerequisite:** the presented card MUST be a multi-channel ISO/IEC 7816-4 card. If it is not, an `InvalidCardResponseException` is raised at scenario execution time.
Parameters	`reader` (`CardReader`) — the reader through which the card is reached. `channelSelectionPolicy` (`ChannelSelectionPolicy`) — the policy governing the channels on which selections are placed.
Returns	A `CardSelectionResult`.
Throws	`ReaderCommunicationException` — if communication with the reader has failed. `CardCommunicationException` — if communication with the card has failed. `InvalidCardResponseException` — if the card does not support multi-channel or returned invalid data.

Schedule Card Selection Scenario

Signature	`scheduleCardSelectionScenario(`
	`  observableCardReader: ObservableCardReader,`
	`  cardPresenceNotificationPolicy: CardPresenceNotificationPolicy,`
	`  selectionExecutionPolicy: SelectionExecutionPolicy`
	`) → void`

Since	3.0
Description	Schedules the execution of the prepared selection scenario as soon as a card is presented to the supplied observable reader. Events are pushed to the registered event handler according to the requested notification policy. The result of the execution MUST be parsed via `parseScheduledCardSelectionsResponse`.
Parameters	`observableCardReader` (`ObservableCardReader`) — the reader through which the card is reached. `cardPresenceNotificationPolicy` (`CardPresenceNotificationPolicy`) — the desired card-presence notification policy. `selectionExecutionPolicy` (`SelectionExecutionPolicy`) — the policy governing the iteration over selection cases.
Throws	None.

5.2.7. Card Selection Result

Name	`CardSelectionResult`
Kind	Interface
Namespace	`reader.selection`
Since	1.0
Purpose	Result of a selection process.

Each selection case prepared with the manager is associated with the index returned by `prepareSelection`. The same index is used to retrieve the corresponding result here. In single-channel mode, at most one case will correspond to the active selected card. In multi-channel mode, several active selections may coexist; see `getActiveSelectionIndexes`.

Get Active Selection Index

Signature	`getActiveSelectionIndex() → integer`
Since	1.0
Description	Returns the index of the active selection, if any. Identical to the first element of `getActiveSelectionIndexes()` when applicable.

Returns	A non-negative `integer` if an active selection exists, `-1` otherwise.
Throws	None.

Get Active Selection Indexes

Signature	`getActiveSelectionIndexes() → list<integer>`
Since	3.0
Description	Returns the indexes of all active selections, one per channel in multi-channel mode.
Returns	A possibly empty list of non-negative `integer` values.
Throws	None.

Get Active Smart Card

Signature	`getActiveSmartCard() → SmartCard?`
Since	1.0
Description	Returns the active matching card, i.e. the card that has been selected. In case of multiple active smart cards (multi-channel mode), the active smart card placed on channel `0` is returned. Use `getActiveSelectionIndexes` to obtain the full set of active selection indexes.
Returns	The active `SmartCard`, or `null` if there is no active card.
Throws	None.

Get Card Type

Signature	`getCardType() → CardType`
Since	3.0
Description	Returns the type of the detected card, as defined by the `CardType` enumeration of the Terminal Definitions API (see CNA-TD-API). The returned value is intrinsically tied to the lifecycle of this `CardSelectionResult`.
Returns	A `CardType` (CNA-TD-API). `UNKNOWN` is returned when the card type could not be identified.
Throws	None.

Get Smart Cards

Signature	`getSmartCards() → map<integer, SmartCard>`
Since	1.0
Description	Returns all `SmartCard` instances corresponding to successful selection cases, indexed by the value returned by `prepareSelection`.
Returns	A possibly empty map.
Throws	None.

5.2.8. Card Selector

Name	`CardSelector`
Kind	Interface
Namespace	`reader.selection`

Generics	`<T extends CardSelector<T>>`
Since	2.0
Purpose	Base contract for all card selectors. Defines filters used to restrict the selection process to a subset of cards.

All filters defined by this interface are **optional** and MAY be combined. If no filter is specified, any card that responds when inserted in the reader is considered selected. Conversely, if one or more filters are defined, a card is not selected if at least one of them rejects it.

Filter By Card Type

Signature	`filterByCardType(cardType: CardType) → T`
Since	3.0
Description	Restricts the selection process to cards whose detected type matches the provided value. The `CardType` enumeration is defined in the Terminal Definitions API (see CNA-TD-API) and characterises the detected card as a combination of transport protocol family and, when applicable, product identity. The value `UNKNOWN` MAY be used to explicitly capture cards whose type could not be identified by the implementation.
Parameters	`cardType` (`CardType` (CNA-TD-API)) — the `CardType` (CNA-TD-API) to use as filter.
Returns	The current instance (`T`).
Throws	None.

Filter By Power On Data

Signature	`filterByPowerOnData(powerOnDataRegex: string) → T`
Since	2.0
Description	Restricts the selection process to cards whose power-on data, as provided by the reader, matches a specific regular expression.
Parameters	`powerOnDataRegex` — the regular expression to use as filter.
Returns	The current instance (`T`).
Throws	`IllegalArgumentException` — if the provided regular expression is empty or invalid.

5.2.9. ISO Card Selector

Name	`IsoCardSelector`
Kind	Interface
Namespace	`reader.selection`
Extends	`CardSelector<IsoCardSelector>`
Since	2.0
Purpose	ISO/IEC 7816-4 card selector. An instance is obtained via `ReaderApiFactory.createIsoCardSelector`.

`IsoCardSelector` adds the ISO/IEC 7816-4-specific filters used to restrict the selection process to certain cards. Like every selector filter, they are all **optional** and MAY be combined. If no filter is specified, any card that responds when presented to the reader is considered selected. Conversely, when one or more filters are defined, the card is not selected as soon as one of them rejects it.

Filter By DF Name (Byte Array)

Signature	`filterByDfName(aid: byte array) → IsoCardSelector`
Since	2.0
Description	Selects a card application DF by its name. The DF is selected only if its name starts with the provided AID, as defined by ISO/IEC 7816-4 § 4.2. The provided AID is used as a parameter of the **Select Application** ISO card command.
Parameters	`aid` — the AID as a `byte array` containing 5 to 16 bytes.
Returns	The current instance (`IsoCardSelector`).
Throws	`IllegalArgumentException` — if the provided array is out of range.

Filter By DF Name (String)

Signature	`filterByDfName(aid: string) → IsoCardSelector`
Since	2.0
Description	Selects a card application DF by its name. The AID is provided as a hexadecimal string. Semantics are otherwise identical to the byte-array overload.
Parameters	`aid` — the AID as a hexadecimal `string` of 5 to 16 bytes.
Returns	The current instance (`IsoCardSelector`).
Throws	`IllegalArgumentException` — if the provided string is invalid or out of range.

Set File Control Information

Signature	`setFileControlInformation(`
	`  fileControlInformation: FileControlInformation`
	`) → IsoCardSelector`

Since	2.0
Description	Sets the file control mode (see ISO/IEC 7816-4). The default value is `FCI`.
Parameters	`fileControlInformation` (`FileControlInformation`) — the file control mode to apply.
Returns	The current instance (`IsoCardSelector`).
Throws	None.

Set File Occurrence

Signature	`setFileOccurrence(fileOccurrence: FileOccurrence) → IsoCardSelector`
Since	2.0
Description	Sets the file occurrence mode (see ISO/IEC 7816-4). The default value is `FIRST`.
Parameters	`fileOccurrence` (`FileOccurrence`) — the navigation option to apply.
Returns	The current instance (`IsoCardSelector`).
Throws	None.

5.2.10. Observable Card Reader

Name	`ObservableCardReader`
Kind	Interface
Namespace	`reader`

Extends	`CardReader`
Since	1.0
Purpose	A `CardReader` capable of observing the insertion and removal of cards.

The observation contract is co-located with the start of detection: a single call to `startCardDetection` provides both the `CardDetectionSettings` that govern the polling behaviour and the `CardReaderEventHandler` that receives the resulting events.

The reader's complete insertion/processing/removal lifecycle—including when each observation event is produced—is described in Reader observation lifecycle.

End Card Processing

Signature	`endCardProcessing() → void`
Since	3.0
Description	Notifies the reader that the **business processing** of the current card has been completed. Upon receiving this notification, the reader: • performs the **DESELECT** sequence on the card (when applicable to the underlying technology) so that the card is brought to a clean idle state; • transitions to the **waiting-for-card-removal** phase, during which the observation cycle waits for the card to leave the reader's field before becoming able to detect a new card. In addition, the reader releases the references to the `SmartCard` instances held following the last selection (see the `SmartCard` lifecycle). This operation is idempotent: calling it twice on the same card processing cycle has no effect after the first call. It is RECOMMENDED to call `endCardProcessing` systematically (e.g. inside a `finally` block) at the end of a card processing whatever the outcome.
Throws	None.

Start Card Detection

Signature	`startCardDetection(`
	`  settings: CardDetectionSettings,`
	`  eventHandler: CardReaderEventHandler`
	`) → void`

Since	3.0
Description	Activates card detection. Once activated, the application is notified of card events through the registered `CardReaderEventHandler`. The `CardDetectionSettings` configure the detection mode, the targeted RF technologies and, when applicable, the ECP frame to emit at polling startup. Parameters carried by `CardDetectionSettings` that are not supported by the underlying reader (for example an unsupported `RfTechnology`, or an ECP frame on a reader that does not support ECP) are silently ignored. A `WARN`-level message is typically logged in such cases. No exception is thrown.
Parameters	`settings` (`CardDetectionSettings`) —the detection configuration. `eventHandler` (`CardReaderEventHandler`)—the application-side handler that receives reader events and errors.
Throws	None.

Stop Card Detection

Signature	`stopCardDetection() → void`

Since	1.0
Description	Stops card detection. After this operation has returned, the registered event handler does not receive card events until detection is restarted.
Throws	None.

5.2.11. Reader API Factory

Name	`ReaderApiFactory`
Kind	Interface
Namespace	`reader`
Since	2.0
Purpose	Factory used by the application to obtain instances of the public types provided by the API.

The `ReaderApiFactory` is the single entry point through which the application instantiates the public types of the API. It is provided as a single instance whose accessors each return a fresh, fully initialised object.

Create Basic Card Selector

Signature	`createBasicCardSelector() → BasicCardSelector`
Since	2.0
Description	Creates a `BasicCardSelector`, a technology-agnostic selector used to target a card in a selection scenario. Use `createIsoCardSelector` when ISO/IEC 7816-4 (AID-based) selection is required.
Returns	A `BasicCardSelector`.
Throws	None.

Create Card Detection Settings

Signature	`createCardDetectionSettings() → CardDetectionSettings`
Since	3.0
Description	Creates a `CardDetectionSettings` carrying the configuration of a card detection cycle.
Returns	A `CardDetectionSettings`.
Throws	None.

Create Card Selection Manager

Signature	`createCardSelectionManager() → CardSelectionManager`
Since	2.0
Description	Creates a `CardSelectionManager`, the service used to prepare and execute a card selection scenario.
Returns	A `CardSelectionManager`.
Throws	None.

Create ISO Card Selector

Signature	`createIsoCardSelector() → IsoCardSelector`
Since	2.0

Description	Creates an `IsoCardSelector` for ISO/IEC 7816-4 (AID-based) card selection. Use `createBasicCardSelector` for technology-agnostic selection.
Returns	An `IsoCardSelector`.
Throws	None.

Get Card Reader Provider

Signature	`getCardReaderProvider() → CardReaderProvider`
Since	3.0
Description	Returns the `CardReaderProvider`, giving standardised access to the readers available in the execution environment.
Returns	A `CardReaderProvider`.
Throws	None.

5.2.12. Scheduled Card Selections Response

Name	`ScheduledCardSelectionsResponse`
Kind	Interface (marker)
Namespace	`reader.selection`
Since	1.0
Purpose	Carrier for the response of the execution of a scheduled selection scenario, provided by a `CardReaderEvent`.

This interface declares no operation. It conveys the card responses to one or more of the scenario's selection cases (selection step itself plus any subsequent commands). Its content MUST be interpreted through `CardSelectionManager.parseScheduledCardSelectionsResponse`.

5.3. SPI Interfaces

5.3.1. Card Reader Event Handler

Name	`CardReaderEventHandler`
Kind	Interface (SPI)
Namespace	`reader.spi`
Since	3.0
Purpose	Interface that the application MUST implement to receive reader events and observation errors from an `ObservableCardReader`.

This SPI unifies the reception of nominal events (`onReaderEvent`) and of fatal observation errors (`onReaderError`) into a single contract, which the application implements once and registers at the call to `startCardDetection`.

On Reader Error

Signature	`onReaderError(`
	`  context: string,`
	`  readerName: string,`
	`  e: throwable`
	`) → void`

Since	3.0
Description	Called when a fatal error occurs on the observed reader. After the operation returns, the observation process is considered stopped.
Parameters	`context` — the context describing the operation that failed. `readerName` — the name of the reader on which the error occurred. `e` — the original exception that triggered the notification.
Throws	None.

On Reader Event

Signature	`onReaderEvent(cardReaderEvent: CardReaderEvent) → void`
Since	3.0
Description	Called when a reader event occurs. Dispatch is **sequential** and **synchronous** unless a different policy is explicitly documented.
Parameters	`cardReaderEvent` (`CardReaderEvent`) — the event payload.
Throws	None. Any exception thrown by the handler is routed to `onReaderError`.

5.3.2. Card Selection Extension

Name	`CardSelectionExtension`
Kind	Interface (SPI, marker)
Namespace	`reader.selection.spi`
Since	2.0
Purpose	Provided by card extensions to enrich a selection case with additional commands and to interpret the response in order to build the corresponding `SmartCard`.

This interface declares no operation at the API level; the contract between the selection manager and the extension is defined by the card extension specification consuming this SPI.

5.3.3. Card Transaction Manager

Name	`CardTransactionManager`
Kind	Interface (SPI)
Namespace	`reader.transaction.spi`
Since	2.1
Purpose	Root contract common to every card transaction manager exposed by a card extension. Defines the single operation `processCommands`.

To exchange data with a card, the application first prepares the commands to transmit (via the operations of the consuming card extension) and then processes them. The preparation step allows commands to be grouped to minimise network exchanges, which is especially valuable in distributed architectures. The `SmartCard` registered with the manager is updated after each data exchange with the card.

Process Commands

Signature	`processCommands() → void`
Since	3.0
Description	Processes all previously prepared commands. All APDUs corresponding to the prepared commands are sent to the card, their responses retrieved and used to update the `SmartCard` associated with the transaction. For write commands, the `SmartCard` is updated only when the command is successful. The process is interrupted at the first failed command. The underlying logical channel is left open after processing.
Throws	`ReaderCommunicationException` — if a communication error with the reader occurs. `CardCommunicationException` — if a communication error with the card occurs. `InvalidCardResponseException` — if a command returns an unexpected status.

5.3.4. ISO Card Transaction Manager

Name	`IsoCardTransactionManager`
Kind	Interface (SPI)
Namespace	`reader.transaction.spi`
Extends	`CardTransactionManager`
Since	3.0
Purpose	Intermediate contract for ISO/IEC 7816-4 cards. Exposes the on-demand cast to multi-channel.

As Multichannel Card Transaction Manager

Signature	`asMultichannelCardTransactionManager() → MultichannelCardTransactionManager`
Since	3.0
Description	Returns a view of this manager as a `MultichannelCardTransactionManager`, granting access to the multi-channel operations. **Prerequisite:** the underlying ISO/IEC 7816-4 card MUST support multi-channel operation; otherwise an `InvalidCardResponseException` is raised during command processing.
Returns	A `MultichannelCardTransactionManager`.
Throws	`InvalidCardResponseException` — if the card does not support multi-channel.

5.3.5. ISO Smart Card

Name	`IsoSmartCard`
Kind	Interface (SPI)
Namespace	`reader.selection.spi`
Extends	`SmartCard`
Since	2.0
Purpose	ISO/IEC 7816-4 smart card whose communication has been established after a successful selection.

In addition to the inherited power-on data, an `IsoSmartCard` exposes the response to the **Select Application** command. Both the power-on data and the **Select Application** response are optional, but at least one of them is always present.

Get Select Application Response

Signature	`getSelectApplicationResponse() → byte array?`
Since	1.0
Description	Returns the data received from the card in response to the **Select Application** command, including the status word.
Returns	The non-empty response bytes, or `null` if no **Select Application** command has been performed.
Throws	None.

Is Basic Channel

Signature	`isBasicChannel() → boolean`
Since	3.0
Description	Indicates whether this smart card is attached to the basic channel (channel `0`) of the underlying ISO/IEC 7816-4 card, or to an additional logical channel.
Returns	`true` if attached to the basic channel, `false` if attached to an additional logical channel.
Throws	None.

5.3.6. Multichannel Card Transaction Manager

Name	`MultichannelCardTransactionManager`
Kind	Interface (SPI)
Namespace	`reader.transaction.spi`
Extends	`CardTransactionManager`
Since	3.0
Purpose	Multi-channel contract. Exposes the operations to close a logical channel explicitly.

This contract MAY be obtained either:

- through `IsoCardTransactionManager.asMultichannelCardTransactionManager`, for ISO/IEC 7816-4 cards on which multi-channel is an optional capability;
- directly as the declared stereotype of a card extension targeting intrinsically multi-channel cards.

Close Channel

Signature	`closeChannel() → void`
Since	3.0
Description	Closes the underlying logical channel without processing any pending command. This operation is idempotent: calling it on an already-closed channel has no effect.
Throws	`ReaderCommunicationException` — if a communication error with the reader occurs. `CardCommunicationException` — if a communication error with the card occurs.

Process Commands And Close Channel

Signature	`processCommandsAndCloseChannel() → void`
Since	3.0
Description	Processes all previously prepared commands and, upon success, closes the underlying logical channel. If any command fails, the channel is left open, as with `processCommands`.

Throws	`ReaderCommunicationException` — if a communication error with the reader occurs. `CardCommunicationException` — if a communication error with the card occurs. `InvalidCardResponseException` — if a command returns an unexpected status.

5.3.7. Smart Card

Name	`SmartCard`
Kind	Interface (SPI)
Namespace	`reader.selection.spi`
Since	1.0
Purpose	Basic smart card whose communication has been established after a successful selection. The instance is ready to receive APDUs.

Card extensions implement (and may extend) this interface to expose the data they collected during the selection.

Get Power On Data

Signature	`getPowerOnData() → string?`
Since	1.0
Description	Returns the card's power-on data, as collected by the reader at the moment the card was inserted. • For a contact reader, this is the Answer To Reset (ATR) defined by ISO/IEC 7816. • For a contactless reader, the reader decides what this data is. Some contactless readers provide a virtual ATR (partially standardised by the PC/SC standard), while other devices may have their own definition, including for example elements from the anti-collision stage of ISO/IEC 14443 (ATQA, ATQB, ATS, SAK, etc.) or any proprietary definitions. Because this data varies from one reader to another, it is exposed here as a `string` that MAY be either a hexadecimal string or any other relevant representation.
Returns	The non-empty power-on data, or `null` if no power-on data is available.
Throws	None.

Is Active

Signature	`isActive() → boolean`
Since	3.0

Description	Returns whether this smart card is currently active on its logical channel. A smart card is considered **active** between its creation (as the result of a successful selection case) and the occurrence of any of the following events, at which point the reader **deactivates** it (`isActive()` returns `false`) and releases its reference: • a **new selection in single-channel mode** (the previous smart cards become obsolete); • an **explicit channel closure request** through `MultichannelCardTransactionManager.closeChannel` or `processCommandsAndCloseChannel`; • a call to `ObservableCardReader.endCardProcessing`; • an exception indicating that the card is no longer contactable (`CardCommunicationException` or `ReaderCommunicationException`). Until then, the reader holds the references to the `SmartCard` instances produced by the last successful selection. Once a smart card has been deactivated, any further attempt to use it is detected prior to any actual exchange with the card.
Returns	`true` if the smart card is active, `false` otherwise.
Throws	None.

5.4. Enumerations

5.4.1. Card Presence Notification Policy

Name	`CardPresenceNotificationPolicy`
Kind	Enumeration
Namespace	`reader.selection.CardSelectionManager`
Since	3.0
Purpose	Options applied when a card is detected, used to decide which cards trigger event-handler notifications.

In the reader observation lifecycle, this policy determines whether a detected card is treated as processable — see Reader observation lifecycle.

Value	Since	Description
`ALWAYS`	3.0	All cards presented to the reader are notified, regardless of the result of the selection.
`MATCHED_ONLY`	3.0	Only cards that have been successfully selected are notified. The others are ignored.

5.4.2. Card Reader Event Type

Name	`Type`
Kind	Enumeration
Namespace	`reader.CardReaderEvent`
Since	1.0
Purpose	Possible card reader events.

Value	Since	Description
`CARD_INSERTED`	1.0	A card has been inserted, with or without a specific selection.

Value	Since	Description
`CARD_MATCHED`	1.0	A card has been inserted that matches the selection criteria.
`CARD_REMOVED`	1.0	The card has been removed from the reader.
`READER_UNREGISTERED`	3.0	The reader has been unregistered and is no longer usable.

5.4.3. Channel Selection Policy

Name	`ChannelSelectionPolicy`
Kind	Enumeration
Namespace	`reader.selection.CardSelectionManager`
Since	3.0
Purpose	Policy governing the use of the basic channel in a multi-channel selection scenario.

Value	Since	Description
`ALLOW_BASIC_CHANNEL`	3.0	The basic channel (channel `0`) MAY host selection cases in addition to the additional logical channels.
`LOGICAL_CHANNEL_ONLY`	3.0	Selection cases are placed only on additional logical channels; the basic channel is not used.

5.4.4. Detection Mode

Name	`DetectionMode`
Kind	Enumeration
Namespace	`reader.CardDetectionSettings`
Since	3.0
Purpose	Defines the behaviour to apply after a card processing cycle has terminated.

In the reader observation lifecycle, this setting is the guard that decides whether detection resumes after a card-processing cycle—see Reader observation lifecycle.

Value	Since	Description
`REPEATING`	3.0	The reader continues waiting for the next card insertion.
`SINGLE_SHOT`	3.0	The reader stops detection after the current card-processing cycle; a new call to `startCardDetection` is required to detect further cards.

5.4.5. File Control Information

Name	`FileControlInformation`
Kind	Enumeration
Namespace	`reader.selection.IsoCardSelector`
Since	2.0
Purpose	Types of templates returned in response to the **Select Application** command, according to ISO/IEC 7816-4.

Value	Since	Description
`FCI`	2.0	File Control Information.

Value	Since	Description
`FCP`	2.0	File Control Parameters.
`FMD`	2.0	File Management Data.
`NO_RESPONSE`	2.0	No response expected.

5.4.6. File Occurrence

Name	`FileOccurrence`
Kind	Enumeration
Namespace	`reader.selection.IsoCardSelector`
Since	2.0
Purpose	Navigation options through the applications contained on the card according to ISO/IEC 7816-4.

Value	Since	Description
`FIRST`	2.0	First occurrence.
`LAST`	2.0	Last occurrence.
`NEXT`	2.0	Next occurrence.
`PREVIOUS`	2.0	Previous occurrence.

5.4.7. Selection Execution Policy

Name	`SelectionExecutionPolicy`
Kind	Enumeration
Namespace	`reader.selection.CardSelectionManager`
Since	3.0
Purpose	Policy governing the iteration over the selection cases of a scenario.

Value	Since	Description
`STOP_ON_FIRST_MATCH`	3.0	The manager stops at the first successful selection case.
`PROCESS_ALL`	3.0	The manager processes every selection case regardless of intermediate successes.

5.5. Exceptions

5.5.1. Card Communication Exception

Name	`CardCommunicationException`
Kind	Runtime exception
Namespace	`reader`
Since	1.0
Purpose	Indicates that the communication with the card has failed, typically because the card was removed from the reader before the exchange could complete.

5.5.2. Invalid Card Response Exception

Name	`InvalidCardResponseException`
Kind	Runtime exception
Namespace	`reader`
Since	2.1
Purpose	Indicates that a response received from the card was invalid—typically when the returned status word does not match any of those expected—either during a selection process or during the processing of a transaction. This exception is also raised when an ISO/IEC 7816-4 card does not support a requested multi-channel operation.

5.5.3. Reader Communication Exception

Name	`ReaderCommunicationException`
Kind	Runtime exception
Namespace	`reader`
Since	1.0
Purpose	Indicates that the communication with the reader has failed. The most likely cause is a physical disconnection of the reader, but other technical problems MAY also be at the origin of the failure.